

Technical Architecture & Backend Systems Report

Project: Software Development Cell (SDC) Portal

Component: Backend Core API

Document Version: 1.0 (Final Submission)

Author: Satyam Diwaker

1. Executive Summary

The Software Development Cell (SDC) Portal Backend is a highly scalable, robust, and secure RESTful API designed to serve as the central nervous system for the organization. It manages the entire lifecycle of student developers—from their initial recruitment and onboarding, through team assignments, project tracking, mentor evaluations, to their eventual graduation as alumni.

Built with modern async Python technologies, the system guarantees high performance, strict data validation, and an easily maintainable modular architecture.

2. Technology Stack

The backend leverages a cutting-edge, production-ready stack to ensure reliability and speed:

- **Framework: FastAPI** — Chosen for its exceptional speed (comparable to NodeJS and Go), native asynchronous support, and automatic OpenAPI documentation generation.
- **ORM & Data Validation: SQLAlchemy** — A powerful library combining SQLAlchemy core for database interactions and Pydantic for rigorous request/response data validation.
- **Database Engine: SQLite** (configured via `sdc_portal.db`) for immediate deployment, with full structural compatibility for seamless migration to PostgreSQL or MySQL in production environments.
- **Security & Cryptography: JSON Web Tokens (JWT)** for stateless session management, utilizing `passlib` (bcrypt) for secure password hashing.
- **Server: Uvicorn** — A lightning-fast ASGI web server implementation.

3. System Architecture

The application follows a **Modular Monolith** architectural pattern. Instead of a highly fragmented microservices approach or a tightly coupled traditional monolith, the system is divided into logically independent, self-contained domains.

3.1 Domain-Driven Directory Structure

The core business logic is encapsulated within the `app/src/` directory, broken down by domain:

- `auth/` : Core identity verification, JWT issuing, and dual-mode login handling.
- `users/` , `teams/` , `mentors/` : Identity and Access Management (IAM), representing the human capital of the SDC.
- `projects/` , `tasks/` , `progress/` : The operational engine tracking software development lifecycles, modules, and iterative tasks.
- `notices/` , `notifications/` : The communication layer handling announcements, targeted broadcasts, and real-time alerts.
- `recruitment/` , `interviews/` : The external-facing pipeline for onboarding new talent.
- `analytics/` , `dashboard/` : Aggregation layers providing metrics and key performance indicators (KPIs) to administrators.
- `files/` , `public/` : Media relay and unauthenticated access endpoints.

3.2 Routing & Middleware

- **CORS Middleware**: Explicitly configured to allow secure communication with external frontends (e.g., React/Vue running on `localhost:5173`).
- **Static File Serving**: Native mounting of local static directories (`/static/uploads`) to handle user avatars, resumes, and project showcases without needing external CDNs during development.

4. Comprehensive Database Schema & Models

The database consists of **over 31 highly normalized tables**. Below is a categorical breakdown of the Entity-Relationship ecosystem:

4.1 Identity & Access Management (IAM)

- **users** : The central entity. Tracks `name` , `email` , `role` (Admin, Developer, Mentor), `branch` , academic years, and social URLs. It incorporates logical properties to calculate dynamic states (e.g., active vs passout).
- **teams** , **team_members** , **team_mentors** : Relational tables mapping users to specific operational groups, tracking leadership designations (Lead, Co-lead, Member) and mentor assignments.
- **hall_of_echoes** : An archival table preserving the legacy of retired/passout members.

4.2 Project & Software Lifecycle Management

- **projects** : Tracks project names, descriptions, deadlines, repository URLs, and statuses (DRAFT, LIVE, PENDING, COMPLETED).
- **modules & tasks** : Granular breakdown of project requirements into sprints. `Tasks` track progress, work logs, blocker remarks, and review feedback.
- **project_srs & srs_reviews** : Formalizes the Software Requirements Specification phase, requiring mentor/admin review before project execution.
- **submissions & submission_reviews** : Facilitates the final delivery of code/demos, linking directly to GitHub repositories.
- **progress_reports & progress_feedback** : Enables continuous tracking of team velocity and blocker resolution.

4.3 Recruitment Pipeline

- **recruitment_drives** : Defines specific time-boxed hiring events.
- **applications** : Captures external candidate data, including resume file references (`files` table integration).
- **interviews & interview_feedback** : Schedules technical/HR rounds and records the final hiring decisions.

4.4 Internal Communications

- **notices & notice_targets** : A highly flexible announcement system supporting global broadcasts or targeted messages to specific teams/roles.
- **notice_replies & notice_reactions** : Adds collaborative, social-media-like interaction to announcements.
- **notifications** : Direct, user-specific alerting system for actionable events.

4.5 System & Telemetry

- **files** : A centralized registry mapping physical file hashes/paths to logical records (used for resumes, project thumbnails, etc.).
 - **audit_logs & activities** : Telemetry tables tracking critical entity modifications to ensure strict administrative oversight and compliance.
 - **system_settings** : Key-value pair configuration for runtime application tuning.
-

5. Security & Authentication Mechanisms

5.1 Dual-Mode Authentication

The system implements a sophisticated hybrid authentication mechanism to accommodate both headless API consumers and standardized documentation tools:

1. **JSON Payload Login (/api/v1/auth/login)**: The primary authentication route used by the React/TypeScript frontend, accepting direct JSON structures.
2. **OAuth2 Form Login (/api/v1/auth/swagger-login)**: A strictly compliant OAuth2 form-data endpoint designed exclusively to satisfy Swagger UI's native **Authorize** interface, resolving **422 Unprocessable Entity** conflicts.

5.2 Role-Based Access Control (RBAC)

Dependencies (`deps.py`) are strictly injected into API endpoints to enforce security boundaries:

- **get_current_user** : Validates the JWT token signature and expiration, ensuring the user exists.
 - **get_current_active_admin** : An elevated dependency that explicitly rejects users without the **admin** role, safeguarding critical endpoints like user creation, deletion, and systemic settings.
-

6. Core Background Services

- **Automated Lifecycle Synchronization (sync_clock)**: An internal chronometer service initialized on startup. It acts as a sweeping mechanism that can run scheduled background tasks—such as dynamically marking developers as **PASSOUT** once their graduation year is reached, reducing manual administrative overhead.
-

7. Conclusion

The SDC Portal Backend represents a mature, enterprise-grade architecture. By leveraging FastAPI and SQLAlchemy, the system achieves maximum type safety, superior performance, and auto-generated documentation. Its Domain-Driven Modular Monolith structure ensures that as the Student Developer Club grows, the backend can scale gracefully to support hundreds of members, dozens of concurrent projects, and vast recruitment drives without systemic degradation.